

INNOVATIONS, OPEN PROBLEMS & BEST PRACTICES

AI Memory, Agent State & Conversational Architecture

A comprehensive cross-platform comparison: emerging techniques, unsolved research problems, anti-patterns, and field-tested best practices

- INNOVATION SCAN
- RESEARCH GAPS
- ANTI-PATTERN CATALOG
- 10+ PLATFORMS

PLATFORMS COMPARED:

Claude ChatGPT Gemini Copilot Perplexity Cursor Devin Manus Replit Agent OpenHands mem0 Letta

12+

Emerging Techniques

9

Open Research Problems

15

Anti-Patterns Catalogued

20+

Best Practices

- > Part A: Emerging Innovations (2025-2026)
- > Part B: Open Research Problems
- > Part C: Cross-Platform Capability Matrix
- > Part D: Best Practices & Anti-Pattern Catalog

TABLE OF CONTENTS

Executive Summary	3
Part A: Emerging Innovations (2025-2026)	4
A.1 Memory-Native Models & MemGPT-Class Architectures	4
A.2 Context Caching & Prompt Reuse	4
A.3 Hybrid Vector + Knowledge Graph Retrieval	4
A.4 MCP-Native Memory & Portable Context	5
A.5 Durable Execution for Agents	5
A.6 Sub-Agent Memory Isolation & Context Compaction	5
A.7 Innovation Scorecard by Platform	5
Part B: Open Research Problems	7
B.1 Memory Provenance & Trust Boundaries	7
B.2 Temporal Validity & Memory Decay	7
B.3 Fine-Grained Agent Recovery	7
B.4 Cross-Session Identity & Personalization Drift	8
B.5 Multi-Agent Shared Memory Consistency	8
B.6 Evaluation of Memory Quality	8
B.7 Privacy-Preserving Long-Term Memory	8
B.8 Context Window Economics at Scale	9
B.9 Trace/Reasoning Transparency vs. Security	9
Part C: Cross-Platform Capability Matrix	10
C.1 Memory Architecture Comparison	10
C.2 Session & Context Management	10
C.3 Agent Recovery & Durability	11
C.4 Trace & Observability	11
C.5 Governance & Enterprise Readiness	12
C.6 Cost & Performance Profile	12
Part D: Best Practices & Anti-Pattern Catalog	13
D.1 Best Practices — Memory Architecture	13
D.2 Best Practices — Session & Context	13
D.3 Best Practices — Agent Recovery	13
D.4 Best Practices — Governance & Security	13
D.5 Anti-Pattern Catalog (15 Patterns)	15
D.6 Anti-Pattern Severity Matrix	16
Closing: Decision Framework	18

Executive Summary

This report extends prior architectural analysis with a forward-looking innovation scan, an honest accounting of unsolved research problems, a granular cross-platform comparison across twelve memory/agent platforms, and a consolidated catalog of best practices paired against their corresponding anti-patterns. The objective is to give engineering and product leaders a decision-ready reference for what to adopt now, what to watch, and what mistakes to deliberately avoid.

Headline Findings

Innovation leaders diverge by layer	No single platform leads across memory, context, and agent durability simultaneously. ChatGPT leads consumer memory UX; Cursor and Devin lead agent durability; Gemini leads raw context-window economics via caching.
MCP is becoming the memory interconnect	Model Context Protocol adoption is turning memory into a portable, swappable layer—analogueous to how SQL standardized database access. This is the single most consequential infrastructure shift identified.
Provenance is the unsolved foundation	Almost no production system distinguishes 'user explicitly said X' from 'agent inferred X from a webpage.' This single gap underlies memory poisoning, hallucinated facts, and trust failures.
Anti-patterns are converging, not diverging	Despite different architectures, the same 15 failure modes recur across all platforms studied—suggesting these are structural risks of the paradigm, not implementation bugs.
Cost-per-context-token is now a competitive axis	Context caching (Gemini), prompt caching (Claude/OpenAI), and hierarchical retrieval are converging on the same goal: decouple conversation length from per-message cost.

INFO: How to use this report: Part A identifies what to adopt or pilot now. Part B frames what remains genuinely unsolved—useful for setting realistic expectations with stakeholders. Part C provides side-by-side platform data for build-vs-buy decisions. Part D is a checklist-style reference for architecture reviews.

PART A

Emerging Innovations (2025-2026)

Twelve techniques that have moved from research papers to production systems within the last 12-18 months, ranked by maturity and adoption breadth.

A.1 Memory-Native Models & MemGPT-Class Architectures

The MemGPT pattern—treating the context window as 'RAM' and external storage as 'disk,' with the model issuing explicit memory read/write function calls—has moved from academic prototype (2023) to productized memory layers. Letta (the company that grew out of MemGPT research) now offers this as a hosted service. The key innovation is that the model itself decides what to page in and out, rather than an external retrieval pipeline making that decision blindly.

Implementation	Maturity	Mechanism	Adopters
Letta (MemGPT)	Production	Model-driven memory paging via function calls; persistent agent state across sessions	Letta Cloud, self-hosted
mem0	Production	Layered memory (user/session/agent) with automatic fact extraction and conflict resolution	Used as memory backend by multiple agent frameworks
Claude Memory (consumer)	Production (opt-in)	Background extraction of durable facts; injected at conversation start	Claude.ai
ChatGPT Memory	Production	Explicit + auto-inferred memory store, user-editable	ChatGPT (Plus/Pro/Team)
Anthropic Memory Tool (API)	Beta/Preview	Developer-controlled memory file system the model can read/write via tool calls	Claude API/Agent SDK builders

A.2 Context Caching & Prompt Reuse

When large portions of a prompt (system instructions, project documents, conversation history) remain identical across requests, providers can cache the model's internal key-value representation of that content and skip recomputation. This converts a linear-cost problem into a near-constant one for the cached portion.

Provider	Feature Name	Cache Window	Cost Impact	Notes
Google Gemini	Context Caching API	Up to 1M tokens, TTL-based	Up to ~75% reduction on cached tokens	Explicit cache object created and referenced by ID
Anthropic Claude	Prompt Caching	Up to 200K tokens, 5-min default TTL (extendable)	Up to 90% reduction on cache hits, ~25% premium on cache writes	Automatic cache breakpoints in system prompt/tools
OpenAI	Prompt Caching	Automatic for prompts >1024 tokens	Up to 50% reduction on cached prefix	Automatic, no explicit cache management needed

A.3 Hybrid Vector + Knowledge Graph Retrieval (Graph RAG)

Pure vector similarity search struggles with multi-hop questions ('what did the client say about the budget in relation to their Q3 hiring plan?'). Graph RAG extracts entities and relationships into a knowledge graph, then combines graph traversal with vector search. Microsoft's GraphRAG research (2024) catalyzed broad adoption; by 2025-2026

it appears in Neo4j's GenAI toolkit, LlamaIndex, LangChain, and several enterprise Copilot deployments.

- **Reported accuracy gains:** 40-60% improvement on multi-hop QA benchmarks versus naive RAG in published evaluations.
- **Cost trade-off:** Graph construction (entity/relation extraction) is an upfront LLM-cost investment; payoff comes from improved retrieval precision over time.
- **Production pattern:** Vector search for 'what's similar', graph traversal for 'what's connected'—used together, not as a replacement.

A.4 MCP-Native Memory & Portable Context

Anthropic's Model Context Protocol (MCP), open-sourced in late 2024, is increasingly used not just for tool connections but as a standard interface for memory providers. A user's memory can live in a third-party MCP server and be accessed by any MCP-compatible client—Claude, IDE agents, or custom applications—without re-platforming. This is the most structurally important shift in the space because it decouples memory from any single vendor's infrastructure.

INFO: Why this matters: today, a user's ChatGPT memory, Claude memory, and Cursor project context are three separate islands. MCP-native memory servers point toward a future where a single memory store is queried by whichever assistant the user happens to be using—shifting competition from 'who has my data' to 'who reasons best with it.'

A.5 Durable Execution for Agents

Temporal-style durable execution—where every step of a long-running workflow is automatically checkpointed and replayable—has gone from a niche backend pattern to the default recommendation for production agents performing more than a handful of tool calls. LangGraph's persistence layer (checkpointing to Postgres/SQLite/Redis) brought this pattern natively into the agent-framework world rather than requiring a separate workflow engine.

- **LangGraph checkpointers:** Built-in support for Postgres, SQLite, and in-memory checkpointing with thread-based state resumption—now a standard starting point for new agent projects.
- **OpenAI Agents SDK:** Run-state persistence with explicit handoff objects between agents, plus tracing dashboards for replay.
- **AWS Bedrock AgentCore / Step Functions:** Managed durable execution for agents running inside AWS, with native integration to Bedrock-hosted models.

A.6 Sub-Agent Memory Isolation & Context Compaction

As multi-agent systems mature, a clear pattern has emerged: spin up sub-agents with their own bounded context for a specific task, then return only a compact summary to the orchestrator—rather than sharing one giant context across all agents. This 'context compaction' approach (popularized in agent-building guidance from Anthropic and others through 2025) prevents context pollution and keeps the orchestrator's window focused on decision-making rather than raw tool output.

- **Pattern:** Orchestrator delegates a research task to a sub-agent; sub-agent reads 50 documents but returns a 200-word synthesis; orchestrator's context grows by 200 words, not 50 documents.
- **Automatic compaction:** Some agent harnesses now auto-summarize and prune tool outputs once the context approaches a token threshold, rather than waiting for a hard failure.
- **Trade-off:** Information loss is real—sub-agent summaries can omit details the orchestrator later needs, requiring a 'pull more detail' fallback mechanism.

A.7 Innovation Adoption Scorecard

Maturity ratings: Production (widely deployed) / Beta (available, limited rollout) / Emerging (research-to-early-product) / Not Present

Platform	Memory-Native	Context Caching	Graph RAG	MCP Memory	Durable Exec.	Context Compaction
Claude	Beta (Memory Tool)	Production	Not native	Production (MCP origin)	Via Agent SDK	Production (guidance)
ChatGPT	Production	Production (auto)	Not native	Emerging	Via Agents SDK	Partial
Gemini	Beta	Production (1M ctx)	Via Vertex AI	Emerging	Via ADK	Partial
Copilot	Production (Graph)	Partial	Production (M365 Graph)	Emerging	Via Azure	Partial
Cursor	Partial (.cursorrules)	Provider-dependent	Not native	Production	Production	Production
Devin	Production (session)	Provider-dependent	Not native	Emerging	Production	Production
Replit Agent	Partial	Provider-dependent	Not native	Emerging	Production	Partial
OpenHands	Partial	Provider-dependent	Not native	Emerging	Production	Partial
mem0 (library)	Production (core focus)	N/A	Beta	Production	N/A	N/A
Letta	Production (core focus)	N/A	Beta	Beta	Production	Production

PART B

Open Research Problems

Nine problems that remain genuinely unsolved as of 2025-2026—not for lack of engineering effort, but because they require either new model capabilities, new evaluation methodologies, or unresolved tradeoffs between competing goals.

B.1 Memory Provenance & Trust Boundaries

When a piece of information enters long-term memory, most systems do not robustly track whether it came from an explicit user statement, an AI inference, a tool output, or third-party content encountered during a task. This single gap is the root cause of memory poisoning (malicious content in a webpage becomes a 'remembered fact'), hallucination compounding (an inferred fact gets stored and later treated as confirmed), and user trust erosion when the AI confidently states something it never verified.

OPEN PROBLEM: No production system reviewed implements a full provenance graph for memories. Proposed approaches (cryptographic signing of memory sources, trust-tier scoring) exist in research form but add significant complexity and have not been validated at scale. The tension is between safety (verify everything) and usefulness (an assistant that constantly says 'I'm not sure where I learned this' is annoying).

B.2 Temporal Validity & Memory Decay

Facts about people, projects, and preferences change. 'User works at Company X' may be true for two years and then false. Current memory systems retrieve by semantic similarity and recency-weighting, but few explicitly model validity intervals or detect contradictions between an old memory and new information.

OPEN PROBLEM: Detecting contradiction requires the system to notice that a new statement conflicts with a stored memory—which itself requires retrieval at write-time, not just read-time, doubling retrieval cost. Some systems (mem0, ChatGPT Memory) perform conflict resolution on write, but evaluation of how often this succeeds versus silently creates duplicate or contradictory memories is largely anecdotal rather than benchmarked.

B.3 Fine-Grained Agent Recovery

Durable execution engines solve step-level recovery well: if an agent fails between tool call 5 and tool call 6, it resumes at tool call 6. The unsolved problem is recovery *within* a tool call—e.g., an agent is writing a 2,000-line file and the connection drops at line 1,400. Restarting the whole tool call may duplicate work, produce inconsistent state, or (for non-idempotent operations like sending an email) cause real-world side effects.

OPEN PROBLEM: Idempotency keys help for simple operations but don't generalize to stateful, multi-step tool operations like file edits or multi-query database transactions. Sub-step checkpointing exists but is expensive and rarely implemented outside specialized coding agents (Cursor, Devin) where file-system snapshots provide a partial answer.

B.4 Cross-Session Identity & Personalization Drift

As memory systems accumulate facts about a user over months or years, the assistant's 'model of the user' can drift from who the user currently is—or can over-fit to a narrow slice of interactions (e.g., heavy use for one work project skews the assistant's sense of the user's interests). There's no consensus on how to balance stability (consistent personalization) against adaptability (updating as the user changes).

OPEN PROBLEM: This is partly an evaluation problem: there's no standard benchmark for 'personalization quality over time.' It's also a UX problem—users rarely audit their stored memory profiles, so drift can go unnoticed until the assistant says something that reveals an outdated assumption.

B.5 Multi-Agent Shared Memory Consistency

When multiple agents (planner, coder, reviewer) operate concurrently with access to shared state, classic distributed-systems problems resurface: race conditions on shared memory writes, stale reads by one agent while another is mid-update, and disagreement about 'the current state of the world' between agents that haven't synchronized.

OPEN PROBLEM: Blackboard-pattern shared state with optimistic locking is the common mitigation, but it doesn't solve semantic consistency—e.g., the coder agent and reviewer agent might both have 'correct' but contradictory understandings of the requirements if the planner updated the plan mid-execution. Formal verification approaches from distributed systems haven't been adapted to LLM-agent semantics yet.

B.6 Evaluation of Memory Quality

There is no widely accepted benchmark for 'is this memory system good?' Existing RAG benchmarks measure retrieval accuracy for static document corpora, not the dynamic, evolving, personal nature of conversational memory. Questions like 'did the system remember the right things, forget the right things, and avoid remembering the wrong things' don't have standard metrics.

OPEN PROBLEM: Some labs use synthetic 'needle in a haystack' tests across long contexts, but these test context-window recall, not memory system quality (extraction, storage, retrieval, and decision-of-relevance as a pipeline). Building a longitudinal benchmark requires simulating weeks/months of realistic user interaction—expensive and methodologically difficult.

B.7 Privacy-Preserving Long-Term Memory

Long-term memory creates a growing, detailed dossier on each user—exactly the kind of data store that attracts regulatory scrutiny, breach risk, and subpoena exposure. On-device and federated approaches (storing memory locally, syncing only derived signals) are technically promising but conflict with the cloud-centric architecture of current frontier models, which need the memory in-context to be useful.

OPEN PROBLEM: Differential privacy techniques add noise that can degrade the precision needed for personalization. Homomorphic encryption for retrieval is computationally prohibitive at current scale. The practical industry response so far is policy-based (deletion rights, encryption at rest) rather than architectural (memory that's cryptographically inaccessible even to the provider).

B.8 Context Window Economics at Scale

Context caching reduces cost for repeated prefixes, but power users with large projects (long-running Claude Projects, extensive Cursor codebases) still generate enormous per-request context. As models support 1M+ token windows, the temptation is to 'just put everything in context'—but this reintroduces the cost and latency problems that retrieval was meant to solve, at a larger scale.

OPEN PROBLEM: There's an unresolved architectural question: at what point does retrieval-augmented context beat brute-force long-context, given caching? The answer depends on cache hit rates, which depend on usage patterns that vary enormously between users—making a one-size-fits-all architecture decision genuinely hard.

B.9 Trace/Reasoning Transparency vs. Security

Users and enterprises increasingly want visibility into agent reasoning for trust and debugging (Part 5 of the prior report). But fuller transparency increases prompt-injection and system-prompt-extraction attack surface. Reasoning models that hide chain-of-thought (to prevent gaming and leakage) directly trade off against explainability requirements emerging from the EU AI Act and similar regulation.

OPEN PROBLEM: No platform reviewed has a satisfying resolution: showing reasoning to end users while cryptographically attesting that the *displayed* reasoning matches the *actual* computation (to prevent post-hoc rationalization being mistaken for genuine explanation) is an active research area with no production implementation found.

PART C

Cross-Platform Capability Matrix

Granular, implementation-level comparison across twelve platforms and libraries spanning consumer assistants, coding agents, autonomous agents, and memory infrastructure components used to build custom systems.

C.1 Memory Architecture Comparison

Platform	Memory Model	Storage Backend	User Visibility	Editable	Scope
Claude	Semantic facts + project memory	Anthropic-managed	Settings panel	Yes (delete)	User + Project
ChatGPT	Explicit + inferred facts	OpenAI-managed	Memory manager UI	Yes (view/edit/delete)	User-global + Project
Gemini	Persona config (Gems) + Workspace	Google-managed	Gem settings	Yes	Gem-scoped
Copilot	Org graph + user signals	Microsoft Graph	Limited (admin)	Admin-controlled	Org + User
Perplexity	Minimal (thread-based)	Perplexity-managed	Limited	Limited	Thread
Cursor	Rules files + codebase index	Local + cloud index	Full (it's a file)	Yes (edit file)	Repo/Workspace
Devin	Session + repo knowledge	Cognition-managed	Session view	Limited	Session/Org
Replit Agent	Project context + checkpoints	Replit-managed	Project view	Limited	Project
OpenHands	Workspace files + history	Self-hosted	Full (open source)	Yes (full control)	Workspace
mem0	Layered: user/session/agent	Configurable (Postgres, Qdrant, etc.)	Developer-defined	Full API control	Developer-defined
Letta (MemGPT)	Agent-driven paging, persistent state	Configurable (Postgres default)	Developer/agent UI	Full API control	Per-agent
LangGraph (lib)	Checkpoint-based state	Postgres/SQLite/Redis/Memory	N/A (library)	Full (it's your DB)	Per-thread

C.2 Session & Context Management

Platform	Context Window	Caching	Summarization	Resume Quality
Claude	200K (1M beta for some models)	Prompt caching (up to 90% savings)	Project + conversation summaries	High
ChatGPT	128K (varies by model)	Automatic prompt caching	Memory-based + thread history	High
Gemini	1M (2M in some configs)	Explicit context caching API	Light (large window reduces need)	Medium-High
Copilot	128K (model-dependent)	Partial	Graph-augmented context	Medium

Platform	Context Window	Caching	Summarization	Resume Quality
Perplexity	32K (search-focused)	Limited	Minimal (search-fresh model)	Medium
Cursor	200K+ (provider-dependent)	Provider-dependent	Codebase-aware chunking	High (dev-focused)
Devin	Provider-dependent	Provider-dependent	Session-level summaries	High
Replit Agent	Provider-dependent	Provider-dependent	Checkpoint-based	Medium-High
OpenHands	Provider-dependent	Provider-dependent	Configurable	Medium (self-managed)

C.3 Agent Recovery & Durability

Platform/Tool	Checkpoint Granularity	Idempotency Support	Human-in-Loop Interrupts	Replay/Audit
Claude (Agent SDK)	Step-level (developer-implemented)	Developer responsibility	Supported via tool design	Via developer logging
ChatGPT (Agents SDK)	Run/handoff-level	Partial	Supported	Tracing dashboard
Gemini (ADK)	Step-level	Developer responsibility	Supported	Cloud logging integration
Cursor	File-snapshot-based	Git as natural idempotency layer	Yes (approve diffs)	Git history
Devin	Step + environment snapshot	Partial (sandboxed env)	Yes	Session replay
Replit Agent	Checkpoint commits	Partial	Yes	Checkpoint history
OpenHands	Step-level (configurable)	Developer responsibility	Yes	Full event log (open source)
LangGraph	Graph-node-level (native)	Developer responsibility	Native (interrupt/resume)	Checkpoint history queryable
Temporal	Activity-level (native)	Native (idempotency keys)	Native (signals)	Full event history, native

C.4 Trace & Observability

Platform	Reasoning Visibility	Tool Call Visibility	Export/API Access	OTel Compatible
Claude	Extended thinking (opt-in, summarized)	Tool names + results in UI	Limited (consumer); full via API	Via custom instrumentation
ChatGPT	Hidden for o-series; visible for others	Expandable in UI	Enterprise audit logs	Via custom instrumentation
Gemini	Thinking mode (partial visibility)	Partial	Cloud-native logging	Yes (GCP integration)
Cursor	Full chain visible	Full	Local logs	No native, but logs are local files
Devin	Step timeline	Full	Session export	Partial
Manus	Action stream	Full	Export trace	Partial
OpenHands	Full verbose (open source)	Full	Full JSON export	Yes (self-instrumented)

Platform	Reasoning Visibility	Tool Call Visibility	Export/API Access	OTel Compatible
LangSmith	N/A (observability tool)	Full capture	Full API	Yes (native)
Arize Phoenix	N/A (observability tool)	Full capture	Full API	Yes (native)

C.5 Governance & Enterprise Readiness

Platform	SSO/SAML	RBAC	Data Residency Options	Compliance Certifications	Memory Deletion API
Claude	Yes (Enterprise)	Yes	US/EU	SOC 2 Type II	Yes
ChatGPT	Yes (Enterprise/Team)	Yes	US/EU	SOC 2, supports HIPAA (BAA)	Yes
Gemini	Yes (Workspace)	Yes	Multi-region	SOC 2, ISO 27001, HIPAA	Yes
Copilot	Yes (Microsoft Entra)	Yes (Graph-based)	Multi-region, sovereign clouds	SOC 2, ISO 27001, FedRAMP (Gov)	Via Microsoft Graph admin
Cursor	Yes (Business plan)	Team-level	US (primary)	SOC 2	Limited (local rules files)
Replit	Yes (Teams)	Team-level	US	SOC 2	Limited
OpenHands	Self-managed	Self-managed	Any (self-hosted)	Self-managed	Full (your infrastructure)
mem0 / Letta	Developer-implemented	Developer-implemented	Any (you choose backend)	Depends on backend chosen	Full (your infrastructure)

C.6 Cost & Performance Profile (Qualitative)

Exact pricing changes frequently and is not reproduced here; the table below reflects relative cost-efficiency patterns for long-running, memory-heavy usage as of 2025-2026.

Platform	Long-Conversation Cost Efficiency	Primary Cost Lever	Notes
Claude	High (with prompt caching)	Prompt caching on repeated project context	Best for stable, document-heavy projects
ChatGPT	High (automatic caching)	Automatic caching, memory keeps prompts smaller	Memory reduces need to re-explain context
Gemini	Very high for huge contexts	Explicit context caching, 1M token window	Best when same large context reused often
Cursor/Devin/Agents	Variable (provider pass-through)	Underlying model choice + context compaction	Cost depends heavily on model selection and agent efficiency
Self-hosted (OpenHands, mem0)	Depends entirely on architecture	Your infrastructure + model choice	Most control, most operational responsibility

PART D

Best Practices & Anti-Pattern Catalog

A consolidated, checklist-ready reference. Best practices are organized by architectural concern; each is paired with the anti-pattern it directly counters in the catalog that follows.

D.1 Best Practices — Memory Architecture

- **Tag every memory with provenance** (user-stated / agent-inferred / tool-derived) even if downstream logic doesn't yet use it—retrofitting provenance after the fact requires re-processing the entire memory store.
- **Assign importance scores at write-time**, not retrieval-time, so pruning decisions can be made cheaply later without re-scoring everything.
- **Implement memory TTLs and validity windows from day one**, even if the default TTL is 'forever'—the schema field is cheap now and expensive to add later.
- **Separate episodic, semantic, and procedural memory into distinct stores or namespaces** even if they share infrastructure—this makes targeted pruning, export, and deletion tractable.
- **Run conflict detection on write** for semantic facts (e.g., 'user's employer') by retrieving similar existing memories before inserting new ones.
- **Cap per-user memory volume** with a defined eviction policy (LRU + importance-weighted) rather than allowing unbounded growth.

D.2 Best Practices — Session & Context Management

- **Use layered context assembly**: recent messages verbatim, mid-range rolling summary, long-range retrieved memory—never rely on context window truncation alone.
- **Preserve 'anchor facts' verbatim through summarization**—numbers, dates, names, and explicit decisions should never be paraphrased away.
- **Adopt prompt/context caching wherever the provider supports it**, structuring prompts so static content (system prompt, project docs) precedes dynamic content (current message).
- **Pre-compute and cache the 'project context package'** so conversation resumption is a cache hit, not a cold assembly.
- **Use sub-agent delegation with context compaction** for research-heavy tasks—return summaries to the orchestrator, not raw tool output.
- **Set token budgets per context layer** and truncate lower-priority layers first when the budget is exceeded, rather than failing the request.

D.3 Best Practices — Agent Recovery

- **Use a durable execution engine** (Temporal, LangGraph persistence, or equivalent) for any agent task with more than 2-3 sequential tool calls or expected runtime over 30 seconds.
- **Assign idempotency keys to every external-effect tool call** (emails, writes, API calls)—generate the key before the call so retries are safe.
- **Checkpoint at tool-call boundaries minimum**; for high-value or long-running tool calls, add sub-step checkpoints (e.g., per-file in a multi-file edit).
- **Design compensating actions for irreversible operations** so a failed multi-step task can be rolled back to a consistent state, not left half-applied.
- **Surface partial progress to the user on reconnect** rather than restarting silently—both for trust and to avoid duplicate work being requested.

D.4 Best Practices — Governance & Security

- **Build the deletion cascade before launch:** a single 'delete user' operation must propagate to messages, memories, embeddings, artifacts, and traces across every store.
- **Never store raw tool outputs as memories without a review/extraction step**—this is the primary memory-poisoning vector.
- **Hide system prompts and raw tool parameters from reasoning traces** shown to end users; show tool names and high-level outcomes only.
- **Instrument with OpenTelemetry from the start**, linking every trace to a conversation ID and tenant ID for auditability and incident response.
- **Sample traces for successful runs, retain everything for failures**—this controls storage growth without losing debugging signal where it matters most.
- **Implement tenant isolation at the query level**, not just the application level—every memory/vector query should be scoped by tenant ID in the query itself, not filtered after retrieval.

D.5 Anti-Pattern Catalog

Fifteen recurring failure modes observed across the platforms and architectures surveyed. Each entry links to the best practice above that directly mitigates it.

AP-01 Context Window Dependence	Critical	Mitigated by D.2
Treating the context window as the entire memory system. When history exceeds the window, the oldest content is silently dropped with no summarization or retrieval fallback—users perceive this as the assistant 'forgetting' mid-conversation.		
AP-02 Unlimited Memory Growth	High	Mitigated by D.1
Storing every extracted memory permanently with no TTL, importance scoring, or cap. Vector index grows unbounded, retrieval precision degrades as noise accumulates, and storage costs scale linearly with engagement rather than value.		
AP-03 Stale Memory Retrieval	High	Mitigated by D.1
Memories from months or years ago surface with the same confidence as current information, because recency is under-weighted or validity windows aren't modeled. The assistant confidently states outdated facts (old job, old preferences, old plans).		
AP-04 Memory Poisoning via Tool Output	Critical	Mitigated by D.4 / D.1
Content encountered during a task (a malicious webpage, a manipulated document) is stored as a 'remembered fact' without provenance tracking or review, and later influences unrelated future responses or actions.		
AP-05 Excessive Summarization Loss	Medium	Mitigated by D.2
Aggressive summarization during context compression discards precise figures, names, or explicit decisions ('the budget is exactly \$47,832' becomes 'around \$48K'), producing downstream errors that compound silently.		
AP-06 Trace Explosion	High	Mitigated by D.4
Logging every micro-step of agent reasoning for every run, with no sampling policy. Trace storage grows to terabytes within weeks, queries time out, and the signal-to-noise ratio for debugging actually decreases.		
AP-07 Missing Checkpointing	Critical	Mitigated by D.3
Long-running agent tasks have no intermediate persistence. A failure at step 27 of 30 requires a full restart—potentially re-triggering side effects (duplicate emails, duplicate writes) that already occurred before the failure.		
AP-08 Session-Server Coupling	Medium	Mitigated by D.3
Conversation or agent state lives in a specific server process's memory rather than an external store. A server restart or scale-down event silently destroys active session state with no recovery path.		
AP-09 Artifact Duplication / Version Sprawl	Low-Med	Mitigated by D.1
Regenerating an artifact repeatedly creates dozens of near-identical versions with no deduplication or clear lineage, leading to storage bloat and user confusion about which version is canonical.		

AP-10 Project / Knowledge Fragmentation	Medium	Mitigated by D.2
Organizations create excessive numbers of separate projects or workspaces for what should be one project with internal structure, making cross-project retrieval impossible and scattering institutional knowledge.		
AP-11 No Conflict Resolution on Memory Write	High	Mitigated by D.1
New facts are appended to memory without checking for contradictions with existing memories. The store accumulates contradictory statements (two different 'current employers') and retrieval can surface either one non-deterministically.		
AP-12 Reasoning Trace as System Prompt Leak	Medium	Mitigated by D.4
Exposing full reasoning chains or tool parameters in user-visible traces inadvertently reveals system prompt structure, internal tool names, or sensitive configuration—creating an extraction/injection attack surface.		
AP-13 Non-Idempotent Retry	Critical	Mitigated by D.3
Retrying a failed tool call after a recovery event without an idempotency key causes duplicated side effects—the canonical example being a 'send email' or 'create record' call executing twice.		
AP-14 Deletion Without Cascade	Critical	Mitigated by D.4
A 'delete my data' request removes the user record and recent messages but leaves orphaned embeddings, cached summaries, or trace data in secondary stores—creating compliance exposure that surfaces only during an audit or breach investigation.		
AP-15 Cross-Tenant Query Without Scoping	Critical	Mitigated by D.4
Vector or memory queries that filter by tenant <i>after</i> retrieval rather than scoping the query itself. Under load or with index misconfiguration, this can return another tenant's data into the result set before the filter is applied.		

D.6 Anti-Pattern Severity & Prevalence Matrix

Prevalence reflects how frequently each anti-pattern was observed across the platforms and architectures surveyed in Parts A-C. Detectability indicates how early in development the issue typically surfaces.

Code	Anti-Pattern	Severity	Prevalence	Typical Detection Point
AP-01	Context Window Dependence	Critical	Very High	Production (user complaints)
AP-02	Unlimited Memory Growth	High	High	Production (cost/perf alerts)
AP-03	Stale Memory Retrieval	High	High	Production (user correction)
AP-04	Memory Poisoning	Critical	Medium	Security review / incident
AP-05	Excessive Summarization Loss	Medium	High	QA / user complaint
AP-06	Trace Explosion	High	Medium	Infra cost review
AP-07	Missing Checkpointing	Critical	Medium	First major outage
AP-08	Session-Server Coupling	Medium	Medium	Scaling event / deploy
AP-09	Artifact Duplication	Low-Med	High	Storage audit
AP-10	Project Fragmentation	Medium	High	Org adoption review

Code	Anti-Pattern	Severity	Prevalence	Typical Detection Point
AP-11	No Conflict Resolution	High	Medium	User-reported inconsistency
AP-12	Trace as Prompt Leak	Medium	Low-Medium	Security review
AP-13	Non-Idempotent Retry	Critical	Medium	Incident (duplicate action)
AP-14	Deletion Without Cascade	Critical	Medium	Compliance audit
AP-15	Cross-Tenant Query Scoping	Critical	Low	Security review / pentest

Closing: Decision Framework

A condensed framework for translating this report into architectural decisions.

If you are building on top of an existing platform

- **Choose based on Part C, Section C.1 and C.6:** match memory model and cost profile to your usage pattern (long stable projects favor caching-heavy platforms; highly dynamic personalization favors explicit memory APIs).
- **Treat MCP compatibility (A.4) as a forward-looking requirement,** not a nice-to-have—it reduces future lock-in risk regardless of which platform you start with.
- **Audit the platform's anti-pattern exposure** using D.6 before committing—ask the vendor directly about deletion cascades (AP-14) and tenant isolation (AP-15) if building multi-tenant products on their API.

If you are building a custom memory/agent system

- **Start with the Part D.1-D.4 best practices as a schema and architecture checklist** before writing the first line of retrieval code—provenance, TTL, and tenant scoping are far cheaper to design in than retrofit.
- **Adopt a durable execution engine (A.5) from the first agent with more than one tool call**—the cost of adoption is small relative to the cost of rebuilding recovery logic after an incident.
- **Consider mem0 or Letta as memory infrastructure** rather than building extraction/storage/retrieval from scratch—both encode many of the D.1 best practices by default.
- **Budget research time for the Part B open problems that affect your use case**—particularly B.1 (provenance) and B.7 (privacy) if handling sensitive personal data, and B.3 (fine-grained recovery) if your agents perform irreversible actions.

INFO: Final note: the field is moving quickly enough that specific product features (Part A, Part C) should be re-verified against current documentation before final decisions. The structural findings—provenance gaps, anti-pattern recurrence, and the shift toward portable memory via MCP—are more durable than any individual platform's current feature set and are the recommended basis for long-term architectural bets.